

NEXORA PHASE 4B — TECHNICAL SPECIFICATION

Deduplicated AI Enrichment & RAG Pipeline

Version: 1.0.0 | Date: 2026-06-26

Priority: P0 — SINGLE EMBEDDING ENGINE, NO DUPLICATE GENERATION

1. ARCHITECTURAL PURPOSE

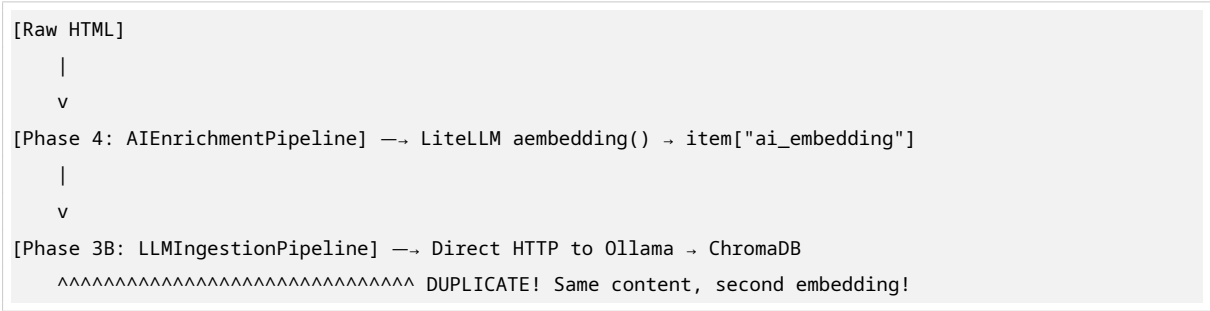
Phase 4B is the **AI enrichment and vector indexing layer**. It consumes clean Markdown from Phase 4A and produces:

- 1. **AI-generated summaries** (2-3 sentences per page)
- 2. **AI-generated tags** (3-5 topic labels per page)
- 3. **Vector embeddings** (one per chunk, for semantic search)
- 4. **Structured chunks** (~512 tokens each, stored in vector DB)

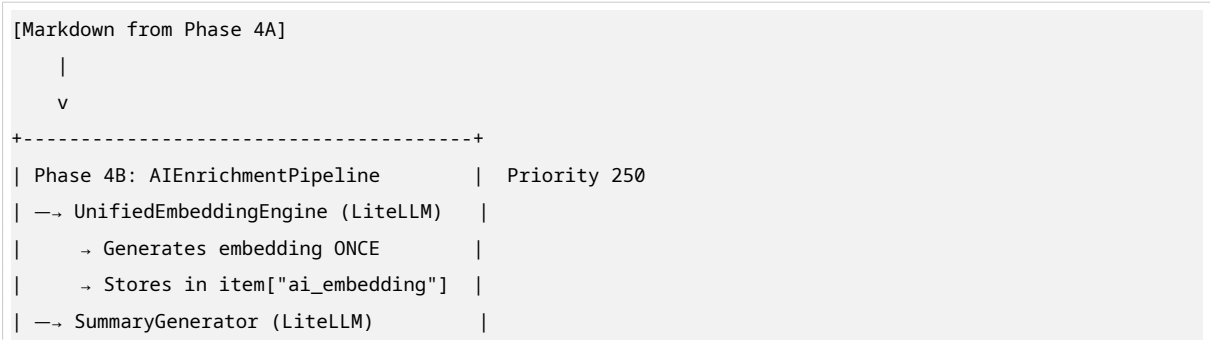
Critical constraint: There is ONE and ONLY ONE embedding generation path in the entire system. The old Phase 3B direct Ollama HTTP call is **eliminated**. All embeddings flow through the unified LiteLLM engine.

2. THE DUPLICATE EMBEDDING PROBLEM (SOLVED)

Old Architecture (BROKEN — do NOT implement)



New Architecture (CORRECT)



```

|   → Stores in item["ai_summary"] |
| → TagGenerator (LiteLLM)         |
|   → Stores in item["ai_tags"]    |
+-----+
|
v
+-----+
| Phase 4B: ChunkingPipeline        | Priority 260
| → Splits markdown into ~512 tokens |
| → Reuses ai_embedding for chunks   |
|   (NO second embedding call!)      |
+-----+
|
v
+-----+
| Phase 4B: VectorIndexPipeline     | Priority 270
| → Stores chunks in ChromaDB        |
| → Links chunks to parent record   |
+-----+

```

3. COMPONENT SPECIFICATIONS

3.1 UnifiedEmbeddingEngine

File: nexora_crawler/ai/embedding_engine.py

Purpose: Single source of truth for ALL embedding generation. No other module calls Ollama directly.

3.1.1 Implementation

```

# embedding_engine.py
# UnifiedEmbeddingEngine — Phase 4B
# SINGLE SOURCE OF TRUTH for all embedding generation.
# Uses LiteLLM for multi-provider support (Ollama, OpenAI, etc.)
# NO direct HTTP calls to Ollama anywhere else in the codebase.

import asyncio
import logging
from typing import List, Optional

from litellm import aembedding

logger = logging.getLogger(__name__)

class UnifiedEmbeddingEngine:
    """
    Unified embedding generator via LiteLLM.

    Supports:
    - Ollama (local): model="ollama/nomic-embed-text"
    """

```

- OpenAI (cloud): model="openai/text-embedding-3-small"
- Any LiteLLM-compatible provider

Usage:

```

engine = UnifiedEmbeddingEngine()
embedding = await engine.embed("text to embed")
embeddings = await engine.embed_batch(["text1", "text2"])
"""

def __init__(
    self,
    provider: str = "ollama",
    model: str = "nomic-embed-text",
    base_url: str = "http://localhost:11434",
    api_key: str = "not-needed",
    timeout: int = 30,
    max_concurrent: int = 3,
):
    self.provider = provider
    self.model = model
    self.base_url = base_url
    self.api_key = api_key
    self.timeout = timeout
    self.semaphore = asyncio.Semaphore(max_concurrent)

    # LiteLLM model string format: "provider/model"
    self.litellm_model = f"{provider}/{model}"

    self.stats = {
        "embeddings_generated": 0,
        "batches_processed": 0,
        "errors": 0,
    }

    async def embed(self, text: str) -> Optional[List[float]]:
        """Embed a single text string. Returns vector or None on failure."""
        if not text or len(text.strip()) < 10:
            return None

        async with self.semaphore:
            try:
                response = await aembedding(
                    model=self.litellm_model,
                    input=text[:8000], # Truncate to safe limit
                    api_base=self.base_url,
                    api_key=self.api_key,
                    timeout=self.timeout,
                )
                embedding = response.data[0]["embedding"]
                self.stats["embeddings_generated"] += 1
                return embedding

```

```

        except Exception as exc:
            logger.warning("[EmbeddingEngine] Failed for text (%d chars): %s",
                           len(text), exc)
            self.stats["errors"] += 1
            return None

    async def embed_batch(self, texts: List[str]) -> List[Optional[List[float]]]:
        """Embed multiple texts concurrently."""
        if not texts:
            return []

        tasks = [self.embed(text) for text in texts]
        results = await asyncio.gather(*tasks, return_exceptions=True)

        embeddings = []
        for result in results:
            if isinstance(result, Exception):
                embeddings.append(None)
                self.stats["errors"] += 1
            else:
                embeddings.append(result)

        self.stats["batches_processed"] += 1
        return embeddings

    def get_stats(self) -> dict:
        return dict(self.stats)

```

3.1.2 Configuration

```

# settings.py
NEXORA_AI_ENABLED = True
NEXORA_AI_PROVIDER = "ollama"           # ollama | openai | anthropic
NEXORA_AI_MODEL = "llama3"             # llama3 | gpt-4o | claude-3-sonnet
NEXORA_AI_EMBEDDING_MODEL = "nomic-embed-text"
NEXORA_AI_BASE_URL = "http://localhost:11434"
NEXORA_AI_API_KEY = "not-needed"
NEXORA_AI_TIMEOUT = 30
NEXORA_AI_MAX_CONCURRENT = 3
NEXORA_EMBEDDINGS_ENABLED = True

```

3.2 AIEnrichmentPipeline (Refactored)

File: nexora_crawler/pipelines/ai_enrichment.py (REFACTORED)

Priority: 250 (after UnifiedSchemaEnricher at 160, before Chunking at 260)

Purpose: Generate summaries, tags, and embeddings via the unified engine.

3.2.1 Implementation

```

# ai_enrichment.py
# AIErichmentPipeline — Phase 4B (REFACTORED)
# Adds semantic summaries, auto-tags, and vector embeddings.
# Uses UnifiedEmbeddingEngine — NO direct Ollama HTTP calls.
# Priority: 250

import asyncio
import json
import logging
from typing import Dict, List, Optional

from litellm import acompletion
from nexora_crawler.ai.embedding_engine import UnifiedEmbeddingEngine

logger = logging.getLogger(__name__)

class AIErichmentPipeline:
    """
    Scrapy pipeline for AI-powered content enrichment.
    Runs at priority 250 (after schema enrichment, before chunking).

    CRITICAL: All embeddings go through UnifiedEmbeddingEngine.
    No direct HTTP calls to Ollama anywhere in this file.
    """

    def __init__(self, crawler):
        self.crawler = crawler
        self.settings = crawler.settings
        self.enabled = self.settings.getbool('NEXORA_AI_ENABLED', False)

        # LiteLLM config for text generation (summary, tags)
        self.provider = self.settings.get('NEXORA_AI_PROVIDER', 'ollama')
        self.model = self.settings.get('NEXORA_AI_MODEL', 'llama3')
        self.base_url = self.settings.get('NEXORA_AI_BASE_URL', 'http://localhost:11434')
        self.api_key = self.settings.get('NEXORA_AI_API_KEY', 'not-needed')
        self.timeout = self.settings.getint('NEXORA_AI_TIMEOUT', 30)
        self.max_concurrent = self.settings.getint('NEXORA_AI_MAX_CONCURRENT', 3)
        self.semaphore = asyncio.Semaphore(self.max_concurrent)

        # Unified embedding engine (SINGLE SOURCE OF TRUTH)
        self.embeddings_enabled = self.settings.getbool('NEXORA_EMBEDDINGS_ENABLED', False)
        if self.embeddings_enabled:
            self.embedding_engine = UnifiedEmbeddingEngine(
                provider=self.provider,
                model=self.settings.get('NEXORA_AI_EMBEDDING_MODEL', 'nomic-embed-text'),
                base_url=self.base_url,
                api_key=self.api_key,
                timeout=self.timeout,
                max_concurrent=self.max_concurrent,

```

```

    )
    else:
        self.embedding_engine = None

    self.stats = {
        "summaries_generated": 0,
        "tags_generated": 0,
        "embeddings_generated": 0,
        "ai_errors": 0,
    }

    @classmethod
    def from_crawler(cls, crawler):
        return cls(crawler)

    async def process_item(self, item, spider):
        if not self.enabled:
            return item

        markdown = item.get("markdown", "")
        if not markdown or len(markdown) < 100:
            return item

        try:
            async with self.semaphore:
                # Run summary, tags, and embedding in parallel
                tasks = []
                tasks.append(self._generate_summary(markdown))
                tasks.append(self._generate_tags(markdown))

                if self.embeddings_enabled and self.embedding_engine:
                    tasks.append(self.embedding_engine.embed(markdown[:4000]))
                else:
                    tasks.append(asyncio.sleep(0))

            summary, tags, embedding = await asyncio.gather(*tasks)

            item["ai_summary"] = summary
            item["ai_tags"] = tags
            if embedding:
                item["ai_embedding"] = embedding
                self.stats["embeddings_generated"] += 1

        except Exception as exc:
            logger.warning("[AI] Enrichment failed for %s: %s",
                           item.get("url", "unknown"), exc)
            self.stats["ai_errors"] += 1

        return item

    async def _generate_summary(self, text: str) -> str:

```

```
        """Generate a 2-3 sentence semantic summary via LiteLLM."""
        prompt = f"""Summarize the following web page content in 2-3 sentences.
Be concise and capture the main points.
```

```
Content:
{text[:4000]}
```

```
Summary: """
```

```
    try:
        response = await acompletion(
            model=f'{self.provider}/{self.model}',
            messages=[{'role': 'user', 'content': prompt}],
            api_base=self.base_url,
            api_key=self.api_key,
            timeout=self.timeout,
            max_tokens=200,
        )
        summary = response.choices[0].message.content.strip()
        self.stats["summaries_generated"] += 1
        return summary
    except Exception as exc:
        logger.warning("[AI] Summary generation failed: %s", exc)
        return ""
```

```
    async def _generate_tags(self, text: str) -> List[str]:
        """Generate 3-5 relevant topic tags via LiteLLM."""
        prompt = f"""Extract 3-5 relevant topic tags from the following content.
Return ONLY a JSON array of strings, no other text.
```

```
Content:
{text[:3000]}
```

```
Tags (JSON array): """
```

```
    try:
        response = await acompletion(
            model=f'{self.provider}/{self.model}',
            messages=[{'role': 'user', 'content': prompt}],
            api_base=self.base_url,
            api_key=self.api_key,
            timeout=self.timeout,
            max_tokens=100,
        )
        content = response.choices[0].message.content.strip()
        if '[' in content and ']' in content:
            start = content.find('[')
            end = content.rfind(']') + 1
            tags = json.loads(content[start:end])
        else:
            tags = [t.strip() for t in content.split(', ')]
```

```

        self.stats["tags_generated"] += 1
        return tags[:5]
    except Exception as exc:
        logger.warning("[AI] Tag generation failed: %s", exc)
        return []

def close_spider(self, spider):
    logger.info("[AI] Pipeline stats: %s", self.stats)
    if self.embedding_engine:
        logger.info("[EmbeddingEngine] Stats: %s", self.embedding_engine.get_stats())

```

3.3 StructuralChunkingPipeline

File: nexora_crawler/pipelines/chunking_pipeline.py

Priority: 260 (after AIErichment at 250, before VectorIndex at 270)

Purpose: Split Markdown into ~512-token chunks with structural awareness.

3.3.1 Implementation

```

# chunking_pipeline.py
# StructuralChunkingPipeline — Phase 4B
# Splits clean Markdown into ~512-token semantic chunks.
# Preserves heading hierarchy, splits at paragraph boundaries.
# Priority: 260

import logging
import re
from typing import List, Dict
from dataclasses import dataclass, field
import uuid

logger = logging.getLogger(__name__)

@dataclass
class NexoraChunk:
    """A single LLM-ready chunk of content."""
    chunk_id: str = field(default_factory=lambda: str(uuid.uuid4()))
    parent_url: str = ""
    parent_title: str = ""
    content: str = ""
    chunk_index: int = 0
    chunk_count: int = 1
    token_count: int = 0
    word_count: int = 0
    heading_chain: List[str] = field(default_factory=list)

    # Inherit from parent record
    ai_summary: str = ""

```



```

ai_tags: List[str] = field(default_factory=list)
# Embedding is generated per-chunk or inherited from parent
embedding: List[float] = field(default_factory=list)

class StructuralChunkingPipeline:
    """
    Scrapy pipeline that chunks Markdown into ~512-token pieces.
    Splits at paragraph boundaries first, then sentences.
    Preserves heading context in each chunk.
    """

    def __init__(self, crawler):
        self.settings = crawler.settings
        self.chunk_size = self.settings.getint('NEXORA_CHUNK_SIZE', 512)
        self.chunk_overlap = self.settings.getint('NEXORA_CHUNK_OVERLAP', 128)

        self.stats = {
            "pages_chunked": 0,
            "chunks_generated": 0,
            "avg_chunk_tokens": 0,
        }

    @classmethod
    def from_crawler(cls, crawler):
        return cls(crawler)

    async def process_item(self, item, spider):
        markdown = item.get("markdown", "")
        if not markdown or len(markdown) < 100:
            item["chunk_count"] = 0
            item["chunk_ids"] = []
            return item

        try:
            chunks = self._chunk_markdown(
                markdown=markdown,
                url=item.get("url", ""),
                title=item.get("title", ""),
                ai_summary=item.get("ai_summary", ""),
                ai_tags=item.get("ai_tags", []),
                ai_embedding=item.get("ai_embedding", []),
            )

            item["chunk_count"] = len(chunks)
            item["chunk_ids"] = [c.chunk_id for c in chunks]
            item["chunks"] = chunks # Store for VectorIndexPipeline

            self.stats["pages_chunked"] += 1
            self.stats["chunks_generated"] += len(chunks)

```

```

        logger.info("[Chunking] %s → %d chunks",
                    item.get("url", "")[:50], len(chunks))

    except Exception as exc:
        logger.error("[Chunking] Failed for %s: %s",
                    item.get("url", ""), exc)
        item["chunk_count"] = 0
        item["chunk_ids"] = []

    return item

def _chunk_markdown(self, markdown: str, url: str, title: str,
                    ai_summary: str, ai_tags: List[str],
                    ai_embedding: List[float]) -> List[NexoraChunk]:
    """
    Split markdown into semantic chunks.

    Strategy:
    1. Split by headings (##, ###) to preserve structure
    2. Within each section, split by paragraphs
    3. If paragraph > chunk_size, split by sentences
    4. Add overlap between chunks
    """
    # Estimate tokens (4 chars ≈ 1 token)
    estimated_tokens = len(markdown) // 4

    if estimated_tokens <= self.chunk_size:
        # Single chunk — no splitting needed
        return [NexoraChunk(
            parent_url=url,
            parent_title=title,
            content=markdown,
            chunk_index=0,
            chunk_count=1,
            token_count=estimated_tokens,
            word_count=len(markdown.split()),
            ai_summary=ai_summary,
            ai_tags=ai_tags,
            embedding=ai_embedding, # Inherit parent embedding
        )]

    # Split by headings first
    heading_pattern = re.compile(r'^({1,6}\s+.+)$', re.MULTILINE)
    sections = heading_pattern.split(markdown)

    chunks = []
    current_chunk = []
    current_tokens = 0
    current_headings = []
    chunk_index = 0

```

```

for section in sections:
    if not section.strip():
        continue

    # Check if this is a heading
    if section.startswith('#'):
        current_headings = self._extract_heading_chain(section)
        continue

    # Split section into paragraphs
    paragraphs = section.split('\n\n')

    for para in paragraphs:
        para = para.strip()
        if not para:
            continue

        para_tokens = len(para) // 4

        if current_tokens + para_tokens > self.chunk_size and current_chunk:
            # Save current chunk
            chunk_text = '\n\n'.join(current_chunk)
            chunks.append(NexoraChunk(
                parent_url=url,
                parent_title=title,
                content=chunk_text,
                chunk_index=chunk_index,
                heading_chain=list(current_headings),
                token_count=len(chunk_text) // 4,
                word_count=len(chunk_text.split()),
                ai_summary=ai_summary,
                ai_tags=ai_tags,
                embedding=ai_embedding,
            ))
            chunk_index += 1

            # Start new chunk with overlap
            overlap_text = self._get_overlap_text(current_chunk, self.chunk_overlap)
            current_chunk = [overlap_text] if overlap_text else []
            current_tokens = len(overlap_text) // 4 if overlap_text else 0

        current_chunk.append(para)
        current_tokens += para_tokens

    # Last chunk
    if current_chunk:
        chunk_text = '\n\n'.join(current_chunk)
        chunks.append(NexoraChunk(
            parent_url=url,
            parent_title=title,
            content=chunk_text,

```

```

        chunk_index=chunk_index,
        chunk_count=len(chunks) + 1,
        heading_chain=list(current_headings),
        token_count=len(chunk_text) // 4,
        word_count=len(chunk_text.split()),
        ai_summary=ai_summary,
        ai_tags=ai_tags,
        embedding=ai_embedding,
    ))

# Update chunk_count for all chunks
for i, chunk in enumerate(chunks):
    chunk.chunk_count = len(chunks)
    chunk.chunk_index = i

return chunks

def _extract_heading_chain(self, heading_text: str) -> List[str]:
    """Extract heading hierarchy from markdown heading."""
    lines = heading_text.strip().split('\n')
    headings = []
    for line in lines:
        match = re.match(r'^#{1,6}\s+(.)$', line.strip())
        if match:
            level = len(match.group(1))
            text = match.group(2).strip()
            headings.append(f"H{level}: {text}")
    return headings

def _get_overlap_text(self, chunks: List[str], overlap_tokens: int) -> str:
    """Get last ~overlap_tokens from previous chunk for context overlap."""
    text = '\n\n'.join(chunks)
    words = text.split()
    overlap_words = words[-min(len(words), overlap_tokens * 3):]
    return ' '.join(overlap_words)

def close_spider(self, spider):
    if self.stats["chunks_generated"] > 0:
        self.stats["avg_chunk_tokens"] = round(
            sum(c.token_count for c in getattr(spider, '_chunks', []))
            / self.stats["chunks_generated"], 1
        )
    logger.info("[Chunking] Pipeline stats: %s", self.stats)

```

3.3.2 Configuration

```

# settings.py
NEXORA_CHUNK_SIZE = 512          # Target tokens per chunk
NEXORA_CHUNK_OVERLAP = 128      # Overlap tokens between chunks

```

3.4 VectorIndexPipeline

File: nexora_crawler/pipelines/vector_index_pipeline.py

Priority: 270 (after Chunking at 260, before Parquet at 450)

Purpose: Store chunks with embeddings in ChromaDB (local) or pgvector (cloud).

3.4.1 Implementation

```
# vector_index_pipeline.py
# VectorIndexPipeline — Phase 4B
# Stores chunks with embeddings in vector database.
# Uses ChromaDB for local, pgvector for cloud (via env switch).
# Priority: 270

import os
import logging
from typing import List, Optional, Dict

from nexora_crawler.pipelines.chunking_pipeline import NexoraChunk

logger = logging.getLogger(__name__)

class BaseVectorStore:
    """Abstract base for vector stores."""

    def add_chunks(self, chunks: List[NexoraChunk]) -> bool:
        raise NotImplementedError

    def search(self, query: str, n_results: int = 10,
               filters: Optional[Dict] = None) -> List[tuple]:
        raise NotImplementedError

    def get_stats(self) -> Dict:
        raise NotImplementedError

class ChromaVectorStore(BaseVectorStore):
    """Local ChromaDB vector store."""

    def __init__(self, collection_name: str = "nexora_chunks",
                 persist_directory: str = "./data/vector_db"):
        import chromadb
        from chromadb.config import Settings

        os.makedirs(persist_directory, exist_ok=True)
        self.client = chromadb.PersistentClient(
            path=persist_directory,
            settings=Settings(anonymized_telemetry=False),
        )
        self.collection_name = collection_name
        self.collection = self._get_or_create_collection()
```

```

def _get_or_create_collection(self):
    try:
        return self.client.get_collection(self.collection_name)
    except ValueError:
        return self.client.create_collection(
            name=self.collection_name,
            metadata={"hnsw:space": "cosine"},
        )

def add_chunks(self, chunks: List[NexoraChunk]) -> bool:
    if not chunks:
        return True

    # Only store chunks that have embeddings
    chunks_with_emb = [c for c in chunks if c.embedding]
    if not chunks_with_emb:
        logger.warning("[VectorStore] No chunks with embeddings to store")
        return False

    ids = [c.chunk_id for c in chunks_with_emb]
    embeddings = [c.embedding for c in chunks_with_emb]
    documents = [c.content[:5000] for c in chunks_with_emb]
    metadatas = [{
        "parent_url": c.parent_url,
        "parent_title": c.parent_title[:200],
        "chunk_index": c.chunk_index,
        "chunk_count": c.chunk_count,
        "token_count": c.token_count,
        "word_count": c.word_count,
        "heading_chain": " | ".join(c.heading_chain)[:500],
        "ai_summary": c.ai_summary[:500],
        "ai_tags": ", ".join(c.ai_tags[:10]),
    } for c in chunks_with_emb]

    self.collection.add(
        ids=ids,
        embeddings=embeddings,
        documents=documents,
        metadatas=metadatas,
    )

    logger.info("[VectorStore] Added %d chunks (total: %d)",
        len(chunks_with_emb), self.collection.count())
    return True

def search(self, query: str, n_results: int = 10,
            filters: Optional[Dict] = None) -> List[tuple]:
    where = None
    if filters:
        where = {}

```

```

        for key, value in filters.items():
            where[key] = {"$eq": value}

    results = self.collection.query(
        query_texts=query,
        n_results=n_results,
        where=where,
        include=["metadatas", "documents", "distances"],
    )

    chunks = []
    if results["ids"]:
        for i, chunk_id in enumerate(results["ids"][0]):
            score = 1.0 - results["distances"][0][i]
            meta = results["metadatas"][0][i]
            doc = results["documents"][0][i]

            chunk = NexoraChunk(
                chunk_id=chunk_id,
                parent_url=meta.get("parent_url", ""),
                parent_title=meta.get("parent_title", ""),
                content=doc or "",
                chunk_index=meta.get("chunk_index", 0),
                chunk_count=meta.get("chunk_count", 1),
                token_count=meta.get("token_count", 0),
                word_count=meta.get("word_count", 0),
                heading_chain=meta.get("heading_chain", "").split(" | "),
                ai_summary=meta.get("ai_summary", ""),
                ai_tags=meta.get("ai_tags", "").split(",") if meta.get("ai_tags") else [],
            )
            chunks.append((chunk, score))

    return chunks

def get_stats(self) -> Dict:
    return {"total_chunks": self.collection.count()}

class VectorIndexPipeline:
    """
    Scrapy pipeline that indexes chunks into vector store.
    Priority: 270
    """

    def __init__(self, crawler):
        self.settings = crawler.settings
        self.enabled = self.settings.getbool('NEXORA_VECTOR_INDEX_ENABLED', True)

        # Choose vector store backend
        backend = self.settings.get('NEXORA_VECTOR_BACKEND', 'chromadb')
        if backend == 'chromadb':

```

```

        self.vector_store = ChromaVectorStore(
            collection_name='nexora_chunks',
            persist_directory=self.settings.get('NEXORA_VECTOR_DB_PATH', './data/vector_db'),
        )
    else:
        raise ValueError(f"Unknown vector backend: {backend}")

    self.stats = {
        "chunks_indexed": 0,
        "pages_indexed": 0,
    }

    @classmethod
    def from_crawler(cls, crawler):
        return cls(crawler)

    async def process_item(self, item, spider):
        if not self.enabled:
            return item

        chunks = item.get("chunks", [])
        if not chunks:
            return item

        try:
            success = self.vector_store.add_chunks(chunks)
            if success:
                self.stats["chunks_indexed"] += len(chunks)
                self.stats["pages_indexed"] += 1
                item["has_embedding"] = True
                item["vector_db_collection"] = "nexora_chunks"
        except Exception as exc:
            logger.error("[VectorIndex] Failed for %s: %s",
                          item.get("url", ""), exc)

        return item

    def close_spider(self, spider):
        logger.info("[VectorIndex] Stats: %s", self.stats)
        logger.info("[VectorStore] %s", self.vector_store.get_stats())

```

3.4.2 Configuration

```

# settings.py
NEXORA_VECTOR_INDEX_ENABLED = True
NEXORA_VECTOR_BACKEND = 'chromadb' # chromadb | pgvector
NEXORA_VECTOR_DB_PATH = './data/vector_db'

# Pipeline registration
ITEM_PIPELINES = {
    # ... Phase 4A pipelines ...

```



```
'nexora_crawler.pipelines.ai_enrichment.AIEnrichmentPipeline': 250,
'nexora_crawler.pipelines.chunking_pipeline.StructuralChunkingPipeline': 260,
'nexora_crawler.pipelines.vector_index_pipeline.VectorIndexPipeline': 270,
'nexora_crawler.pipelines.parquet_export.ParquetExportPipeline': 450,
# ... rest ...
}
```

4. EMBEDDING DEDUPLICATION GUARDRAIL

4.1 The Rule

RULE: `UnifiedEmbeddingEngine.embed()` is the **ONLY** function in the entire codebase that generates embeddings. No other module, pipeline, script, or utility may call `aembedding()` directly, make HTTP requests to `localhost:11434`, or use any other embedding API.

4.2 Enforcement

```
# In code review, check for these banned patterns:
BANNED_PATTERNS = [
    "httpx.post.*11434",          # Direct Ollama HTTP
    "requests.post.*11434",      # Direct Ollama HTTP
    "aembedding\" ,              # Direct LiteLLM aembedding (outside engine)
    "ollama.embed",              # Ollama Python client
]

# The ONLY allowed pattern:
ALLOWED_PATTERN = "UnifiedEmbeddingEngine.embed"
```

4.3 Migration from Old Phase 3B

If old `llm_ingestion_pipeline.py` exists:

1. **Delete** the file entirely
2. **Delete** `storage/vector_store.py` (old ChromaDB wrapper)
3. **Delete** `storage/metadata_store.py` (old SQLite wrapper — replaced by Phase 4A)
4. **Keep** `storage/models.py` but update to use `NexoraChunk` from `chunking_pipeline`

5. TEST MATRIX

Test ID	Scenario	Expected Result
P4B-T01	<code>UnifiedEmbeddingEngine.embed()</code>	Returns 384-dim vector for <code>nomic-embed-text</code>
P4B-T02	<code>UnifiedEmbeddingEngine.embed_batch()</code>	Returns list of vectors, handles failures gracefully
P4B-T03	AI summary generation	<code>ai_summary</code> is 2-3 coherent sentences
P4B-T04	AI tag generation	<code>ai_tags</code> is list of 3-5 relevant strings
P4B-T05	No duplicate embeddings	Only ONE embedding generated per page
P4B-T06	Chunking	Markdown split into 400-600 token chunks

Table 1 – continued

Test ID	Scenario	Expected Result
P4B-T07	Chunk overlap	Adjacent chunks share ~128 tokens of context
P4B-T08	Heading preservation	Chunks retain heading hierarchy in metadata
P4B-T09	Vector store insert	Chunks stored in ChromaDB, count increases
P4B-T10	Semantic search	Query returns relevant chunks with similarity scores
P4B-T11	Multi-provider	Switching Ollama→OpenAI works via config change only
P4B-T12	No regression	Phase 3 + Phase 4A tests still pass

6. DEFINITION OF DONE

- `UnifiedEmbeddingEngine` is the ONLY embedding generator in the codebase
- All old Phase 3B embedding code is deleted
- `AIEnrichmentPipeline` uses `UnifiedEmbeddingEngine` for embeddings
- `StructuralChunkingPipeline` splits Markdown into ~512-token chunks
- `VectorIndexPipeline` stores chunks in ChromaDB with embeddings
- Semantic search returns relevant results on test queries
- Multi-provider switching works (Ollama OpenAI)
- No duplicate embedding generation anywhere in the system
- All 12 test cases pass
- Phase 3 + Phase 4A tests show no regression